

Market Data Hub Refactoring Analysis

Current Architecture Issues

Data Fetching Redundancy

python

```
# Currently in your bots:  
# JIT bot: client.get_orderbook() every 0.9s  
# Hedge bot: client.get_orderbook() every 5s  
# Trend bot: client.get_orderbook() every 3s  
# = ~3x redundant calls to same endpoints
```

Scattered Feature Computation

python

```
# JIT bot calculates microprice  
mid = (ob.bids[0][0] + ob.asks[0][0]) / 2.0  
# JIT bot calculates spread  
spread_bps = ((ask - bid) / mid) * 10000  
  
# Trend bot calculates volatility  
volatility = calculate_volatility(prices)  
  
# Each bot does similar math independently
```

MDHub Transformation Benefits

1. Centralized Data Ingestion

yaml

```
Before: 3 bots → 3 connections → Drift API  
After: MDHub → 1 connection → Drift API  
3 bots → local WebSocket → MDHub
```

2. Shared Feature Engineering

python

MDHub computes once, all bots consume:

```
{
  "symbol": "SOL-PERP",
  "mid": 150.234,
  "spread_bps": 2.1,
  "microprice": 150.237, # OBI-weighted
  "imbalance": 0.15,    # bid/ask imbalance
  "volatility_1m": 0.0023,
  "last_trade": {...}
}
```

3. Consistent Market State

python

All bots see EXACTLY the same market snapshot

No more timing skew between bot decisions

timestamp = 1693420800.123

all_bots_get_same_data_at_timestamp()



Refactoring Scope & Impact

HIGH IMPACT Changes

Component	Current	After MDHub	Effort
Bot Data Access	client.get_orderbook()	mdhub_client.get_snapshot()	Medium
Client Library	libs/drift/client.py	libs/mdhub/client.py	High
Feature Engineering	Scattered in each bot	Centralized in MDHub	High
Configuration	Single bot configs	Bot + MDHub configs	Low
Deployment	1 service (orchestrator)	2 services (orchestrator + MDHub)	Medium

MEDIUM IMPACT Changes

File	Changes Needed	Complexity
run_all_bots.py	Replace client initialization	Low
bots/jit/main.py	Remove microprice/OBI calc, use MDHub data	Medium
bots/hedge/main.py	Remove volatility calc, use MDHub data	Medium
bots/trend/main.py	Remove price history, use MDHub indicators	Medium
configs/	Add MDHub endpoint configs	Low

Refactoring Strategy

Phase 1: Add MDHub Alongside (2-3 days)

```
bash

# Deploy MDHub as separate service
uvicorn services.mdhub.main:app --port 8088

# Bots still use old client, but MDHub running in parallel
# Validate MDHub data quality vs direct Drift calls
```

Phase 2: Migrate JIT Bot First (1-2 days)

```
python

# Replace in bots/jit/main.py:
- ob = await client.get_orderbook()
- mid = (ob.bids[0][0] + ob.asks[0][0]) / 2.0
+ snapshot = await mdhub_client.get_snapshot("SOL-PERP")
+ mid = snapshot.mid
+ microprice = snapshot.microprice # Already computed!
```

Phase 3: Migrate Hedge & Trend (1-2 days)

```
python

# Hedge bot gets pre-computed volatility
volatility = snapshot.volatility_1m # vs calculating from price history

# Trend bot gets pre-computed indicators
macd_signal = snapshot.macd # vs maintaining price deque
```

Phase 4: Remove Old Client (1 day)

```
python

# Clean up libs/drift/client.py redundant methods
# All market data now flows through MDHub
```

Bonus Improvements You Get

Real-time Streaming

```
python

# Bots can subscribe to live updates
async for update in mdhub_client.stream("ticks", "SOL-PERP"):
    if update.type == "trade":
        handle_new_trade(update)
    elif update.type == "l2_update":
        handle_orderbook_update(update)
```

Historical Data

```
python

# Easy backtesting with consistent data
trades = mdhub_client.get_trades("SOL-PERP", since=yesterday)
backtest_strategy(trades)
```

Multiple Markets

```
python

# Scale to multiple symbols with no bot changes
mdhub_client.subscribe(["SOL-PERP", "ETH-PERP", "BTC-PERP"])
```

Migration Risks & Mitigations

Risk: Network Dependency

Problem: Bots now depend on MDHub being online **Mitigation:**

- Health checks and auto-restart
- Local snapshot caching in bots
- Circuit breaker to fallback mode

Risk: Single Point of Failure

Problem: MDHub down = all bots down **Mitigation:**

- Deploy MDHub with high availability

- Keep fallback to direct Drift client in emergency

Risk: Added Latency

Problem: Network hop from bot → MDHub → Drift **Mitigation:**

- Deploy MDHub on same server/network
- Use local Redis for <1ms latency
- WebSocket streaming vs polling

Cost/Benefit Analysis

Costs (One-time)

- **Development:** ~5-7 days of refactoring
- **Testing:** Validation against current bot performance
- **Deployment:** Docker compose complexity, monitoring setup

Benefits (Ongoing)

- **Performance:** 3x fewer API calls to Drift
- **Consistency:** All bots make decisions on same data
- **Scalability:** Add new bots with zero data fetching code
- **Features:** Rich derived data (microprice, imbalance, volatility)
- **Monitoring:** Centralized data quality metrics
- **Backtesting:** Historical data for strategy development

Recommendation: YES, Refactor Now

This is the **perfect time** because:

1. **Clean interfaces:** Your current bot separation makes this easier
2. **Mock client:** You already have abstraction layers to replace
3. **Production-ready:** Your system is already sophisticated enough to benefit
4. **Growth path:** Essential for adding more markets/strategies

The refactoring will transform your system from "hobby project" to "institutional-grade infrastructure" - exactly what you need for serious trading.

Next Steps

1. **Deploy MDHub** alongside current system (no bot changes yet)
2. **Validate data quality** - compare MDHub vs direct Drift calls
3. **Migrate JIT bot first** (highest frequency, biggest performance gain)
4. **Migrate remaining bots** once JIT validation complete
5. **Remove old client** and celebrate your more efficient system!

The architectural upgrade will pay dividends as you scale to more markets and strategies.